

Bitstream Studio

คู่มือการใช้งานฉบับสมบูรณ์

ส่วนขยาย VS Code สำหรับอ่านค่าเซนเซอร์แบบเรียลไทม์ ตั้งค่าอุปกรณ์ สร้างโฟลว์แบบลากวาง และ Digital Twin 3D — สำหรับบอร์ด TESA IoT DevKit และ PSoC Edge (PSoC E84 AI)

เวอร์ชันโปรแกรม **0.1.7** ผู้พัฒนา **TERNIONDEV** โปรไฟล์รุ่นที่ติดตั้ง **minimal-sensor** แคลิเดตาออกเซนเซอร์ **2026-06-10**

จัดทำคู่มือ **11 กรกฎาคม 2026**

สารบัญ

- | | |
|--|--|
| 1. รู้จัก Bitstream Studio | 10. การตั้งค่าเซนเซอร์และคำสั่งควบคุม |
| 2. การติดตั้งและความต้องการระบบ | 11. MQTT / WebSocket Topics |
| 3. เริ่มใช้งานครั้งแรก (Quick Start) | 12. คลังโหนดใน Sensor Studio |
| 4. เวิร์กสเปซทั้งสอง | 13. Digital Twin — โมเดล 3D |
| 5. เซนเซอร์บนบอร์ดทั้ง 4 ตัว | 14. โหมด Simulator |
| 6. กล้องและงาน Vision | 15. คำสั่ง การตั้งค่า คีย์ลัด และพอร์ต |
| 7. การเชื่อมต่ออุปกรณ์ | 16. รุ่นของโปรแกรม (Tier) |
| 8. วิธีเรียกอ่านค่าเซนเซอร์ | 17. การแก้ปัญหา |
| 9. Implement จริง: ส่งข้อมูลจากบอร์ดขึ้นเว็บ | 18. โปรโตคอล BS2 (สำหรับผู้ใช้ขั้นสูง) |
| | 19. แหล่งอ้างอิง |

1 รู้จัก Bitstream Studio

โปรแกรมนี้คืออะไร ทำอะไรได้บ้าง และส่วนประกอบภายในทำงานร่วมกันอย่างไร

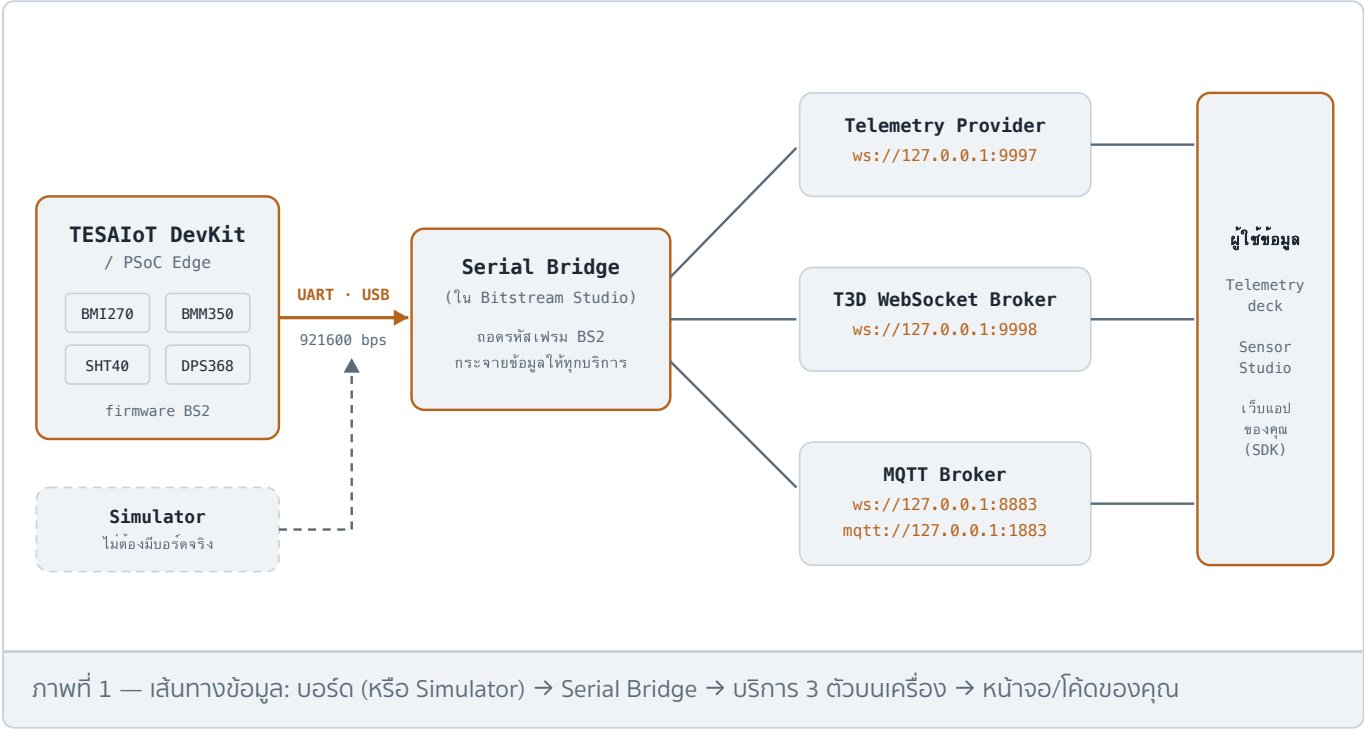
Bitstream Studio คือส่วนขยาย (extension) ของ Visual Studio Code ที่เปลี่ยน VS Code ให้กลายเป็นห้องควบคุมสำหรับงาน IoT — ใช้ตั้งค่าเซนเซอร์ ดูค่าที่สตรีมเข้ามาแบบเรียลไทม์ ตรวจสอบสภาพการเชื่อมต่อ สร้างโปรแกรมแบบลากวางด้วยโหนด (node-based flow) และแสดงผลผ่านโมเดล 3D แบบ Digital Twin ทั้งหมดในหน้าต่างเดียว

ใช้งานได้กับฮาร์ดแวร์จริง 2 แบบ คือบอร์ด **TESA IoT DevKit** และ **PSoC Edge (PSoC E84 AI)** ผ่านสาย USB, Wi-Fi หรือ MQTT — และถ้าไม่มีบอร์ดก็ใช้ **โหมด Simulator** จำลองข้อมูลเซนเซอร์ได้ครบทุกตัว

ความสามารถหลัก

ด้าน	ทำอะไรได้
ตั้งค่าเซนเซอร์	เลือกว่าเซนเซอร์ตัวไหนส่งข้อมูล ส่งที่แคไหน ส่งช่องข้อมูลใดบ้าง และแสดงอะไรบนหน้าจอ
สตรีมเรียลไทม์	ดูค่าอัปเดตสด ๆ จากบอร์ดหรือจากตัวจำลอง (BMI270 ทำได้ราว 25 Hz)
วินิจฉัยระบบ	Activity log, สถานะลิงก์, สุขภาพการเชื่อมต่อ ระหว่างทำงาน
การเชื่อมต่อ	Serial (USB), Wi-Fi และ MQTT ตั้งค่าได้จากเมนู ≡ ในแอป
Digital Twin	โมเดล 3D ได้ตอบโต้ — แขนกลหุ่นยนต์ เครื่องจักร โดรน บอร์ดพัฒนา ฯลฯ ขยับตามค่าเซนเซอร์จริง
Visual Flow	ลากวางโหนดตรรกะ คณิตศาสตร์ ตัวแสดงผล (เกจ กราฟ LED) ต่อสายเข้ากับสตรีมข้อมูลสด

สถาปัตยกรรมของระบบ



2 การติดตั้งและความต้องการระบบ

รายการ	ข้อกำหนด
VS Code	เวอร์ชัน 1.75 ขึ้นไป (ใช้บน Cursor ได้)
ระบบปฏิบัติการ	Windows หรือ macOS
บอร์ด	ไม่บังคับ — มีโหมด Simulator ให้ใช้แทน
Node.js	ไม่จำเป็นสำหรับการใช้งานทั่วไป (จำเป็นเฉพาะเมื่อเขียนเว็บแอปด้วย SDK)
อินเทอร์เน็ต	จำเป็นครั้งแรกครั้งเดียว เพื่อดาว์นโหลดโมเดล 3D

วิธีติดตั้ง

- ติดตั้งจาก Visual Studio Marketplace (ค้นหา "Bitstream Studio") หรือกด **Install** ในแท็บ Extensions ของ VS Code
- หรือติดตั้งผ่านเทอร์มินัล: `code --install-extension TERNIONDEV.bitstream-studio` (บน Cursor ใช้ `cursor` แทน `code`)
- หรือติดตั้งจากไฟล์ `.vsix` โดยตรง: Extensions → เมนู → **Install from VSIX...** แล้วเลือกไฟล์ `bitstream-studio-0.1.7.vsix`

3 เริ่มใช้งานครั้งแรก (Quick Start)

3 ขั้นตอนหลัก

- เปิด Command Palette (`Cmd/Ctrl` + `Shift` + `P`) → **Bitstream Studio: Open Bitstream Studio**
- Command Palette → **Bitstream Studio: Start Serial Bridge**
- ที่แถบเครื่องมือ (toolbar) เลือกแหล่งข้อมูล **Bitstream** (บอร์ดจริง) หรือ **Simulator** → กด **Link**

การรันครั้งแรก: โปรแกรมจะดาวน์โหลดโมเดล 3D (แขนกล เครื่องจักร โดรน บอร์ด และจากแวดลอม) จาก GitHub ก่อนที่การ์ดเวิร์กสเปซจะปลดล็อก การเปิดครั้งแรกไปจะข้ามขั้นนี้ ถ้าภาพพร๊ิวว่างเปล่า ให้สั่ง **Bitstream Studio: Sync Free Pack to Disk** (ต้องมีอินเทอร์เน็ตหนึ่งครั้ง ไม่ต้องมีบัญชี GitHub)

ใช้กับบอร์ดจริง

- เสียบบอร์ด TESA IoT DevKit หรือ PSoC Edge เข้ากับคอมพิวเตอร์ผ่าน USB
- สั่ง **Start Serial Bridge** → toolbar เลือก **Bitstream** → กด **Link** (โปรแกรมจะจับมือ HELLO กับ เฟิร์มแวร์อัตโนมัติ)

ใช้โดยไม่มีบอร์ด

- สั่ง **Start Bitstream Simulator** (หรือติดตั้งส่วนขยาย Bitstream Simulator แยกแล้วกด Streaming)
- toolbar เลือก **Simulator** → กด **Link**

ข้อควรรู้: โหมด Bitstream (UART) กับโหมด Simulator ทำงานแทนกัน — เลือกได้ทีละเส้นทางเท่านั้น (mutually exclusive) ทุก sample จะติดป้ายบอกที่มาว่า `origin: "uart"` หรือ `origin: "sim"`

4 เวิร์กสเปซทั้งสอง

รุ่นที่ติดตั้ง (โปรไฟล์ minimal-sensor) เปิดใช้ 2 เวิร์กสเปซ

4.1 Sensor Telemetry — ห้องควบคุมและมอนิเตอร์

- Sensor configuration** — ปรับตั้งค่าเซนเซอร์รายตัว (เปิด/ปิด, ความถี่, โหมดส่ง, ช่องข้อมูล) แล้วกด **Apply**

- **Telemetry deck** — แผงแสดงค่าสดทุกเซนเซอร์ พร้อมเกจและกราฟ
- **3D orientation preview** — ภาพ 3D แสดงการเอียง/หมุนของบอร์ดจากเซนเซอร์การเคลื่อนไหว (BMI270)
- **Activity log** — บันทึกเหตุการณ์ระบบและข้อมูลวินิจฉัย
- **Actuator Config deck** — ควบคุม LED บนบอร์ด (ความสว่างแบบเชิงเส้น 0.01% ต่อขั้น, PWM ~1 kHz)

4.2 Sensor Studio — ห้องทดลองแบบ Visual

- **Node-based flow editor** — ลากโหนดจากพาเนล **Library** มาวางบนแคนवास ต่อสายจากเซนเซอร์ → ตรรกะ/คณิต → ตัวแสดงผล
- **Digital twin views** — โมเดล 3D ของหุ่นยนต์ เครื่องจักร ยานพาหนะ และบอร์ด ขับเคลื่อนด้วยข้อมูลสด
- **Output nodes** — เกจ พล็อตเตอร์ LED และตัวชี้วัดบนหน้าจอแบบอื่น ๆ
- ทำงานได้ทั้งแบบ **2D** (แคนवासโฟลว์, วิดเจ็ต) และ **3D** (twin, ฟรีวิว, โต้ตอบ)

เมนูลัดในหน้า 3D: กดปุ่ม  หรือคลิกขวา ขณะชี้เมาส์อยู่บนแคนवास 3D จะเปิดเมนูตามตำแหน่ง เช่น *Fit to view* และ *Reload scene* ของ Machine Twin — เมนูนี้เปิดเฉพาะเมื่อคอร์เซอร์อยู่บนแคนवास 3D เท่านั้น

5 เซนเซอร์บนบอร์ดทั้ง 4 ตัว

บอร์ดมีเซนเซอร์ 4 ตัว แต่ละตัวมีรหัสประจำ (sensor id) ช่องข้อมูล (field) หน่วย และช่วงค่าที่วัดได้ตามตารางด้านล่าง

bmi270 · id 0

BMI270

IMU 6 แกน — ความเร่ง + ไจโร +
อุณหภูมิ พร้อม Sensor Fusion
(Euler / Quaternion)

bmm350 · id 1

BMM350

สนามแม่เหล็ก 3 แกน (เข็มทิศ) +
อุณหภูมิ

sht40 · id 2

SHT40

อุณหภูมิ + ความชื้นสัมพัทธ์

dps368 · id 3

DPS368

ความดันบรรยากาศ (บารอมิเตอร์) +
อุณหภูมิ — ใช้หาความสูงได้

เข้าใจภาพรวมก่อน: 4 ตัวนี้ต่างกันอย่างไร

เซนเซอร์แบ่งเป็น 2 กลุ่มตามสิ่งที่มันมอง — กลุ่มแรกมองที่**ตัวบอร์ดเอง** (ขยับ เอียง หมุน หันทิศไหน) กลุ่มที่สองมองที่**อากาศรอบบอร์ด** (ร้อน ชื้น ความกดอากาศ):

เซนเซอร์	วัดอะไร (ภาษาคน)	ตอบคำถามว่า	ตัวอย่างการใช้งานจริง
BMI270 กลุ่มการเคลื่อนไหว	ความเร่ง (ถูกเขย่า/เอียงแคไหน — ตัวเดียวกับที่ทำให่มือถือหมุนจอ) + ความเร็วการหมุน + ทำทาง 3D สำเร็จรูปจาก Fusion	"บอร์ดกำลังขยับ เอียง หรือหมุนอยู่ไหม"	ตรวจสอบการสั่นเครื่องจักร, ตรวจสอบการล้ม, ควบคุมโดรน/หุ่นยนต์ทรงตัว, หมุนโมเดล 3D ตามบอร์ด
BMM350 กลุ่มการเคลื่อนไหว	สนามแม่เหล็กโลก = เข็มทิศดิจิทัล และตรวจแม่เหล็ก/มอเตอร์/โลหะใกล้ ๆ ได้ (ค่ากระโดดผิดปกติ)	"หันหน้าไปทิศไหน"	เข็มทิศหุ่นยนต์, ตรวจสอบประตูเปิด-ปิด (แม่เหล็กติดบาน), ตรวจสอบโลหะเข้าใกล้
SHT40 กลุ่มสิ่งแวดล้อม	อุณหภูมิอากาศ + ความชื้นสัมพัทธ์ (ตัวเดียวบนบอร์ดที่วัดอุณหภูมิ ห้อง จริง)	"อากาศร้อนไหม ชื้นไหม"	เทอร์โมมิเตอร์ห้อง, สั่งเปิดพัดลม/แอร์, ฟาร์มอัจฉริยะ, เตือนความชื้นสูงเสี่ยงเชื้อรา
DPS368 กลุ่มสิ่งแวดล้อม	ความดันบรรยากาศ — ความดันลดราว 12 hPa ต่อความสูง 100 m จึงใช้วัดความสูงได้	"ความกดอากาศเท่าไร / อยู่สูงแค่ไหน"	วัดความสูง/ชั้นของตึก, ระดับความสูงโดรน, สังเกตความดันตกเร็ว = ฝนกำลังมา

ทำไมหลายตัวมีค่า "อุณหภูมิ" ซ้ำกัน? BMI270, BMM350 และ DPS368 วัดอุณหภูมิ**ของตัวชิปเอง**ไว้ชดเชยความแม่นยำภายในเท่านั้น (ชิปทำงานจะอุ่นกว่าอากาศ) — ต้องการอุณหภูมิ**ห้อง**จริง ให้ใช้ค่าจาก **SHT40** เสมอ

ไจโรกับเข็มทิศต่างกันอย่างไร? ไจโร (BMI270) บอกได้ว่า "หมุนไป/เพิ่มท่าไหร่จากเดิม" — ตอบสนองเร็วแต่ค่าเพี้ยนสะสมเมื่อเวลาผ่านไป ส่วนเข็มทิศ (BMM350) บอกทิศทางเทียบทิศเหนือได้ตลอดแต่ช้ากว่าและไวต่อโลหะรอบข้าง — ระบบ Fusion ใช้จุดแข็งทั้งคู่ร่วมกันจึงได้ทิศทางที่ทั้งเร็วและตรง

5.1 BMI270 — IMU (ความเร่ง / ไจโร / การหมุน)

เซนเซอร์วัดการเคลื่อนไหวกหลักของบอร์ด นอกจากค่าดิบแล้ว เฟิร์มแวร์ยังคำนวณ **Sensor Fusion** ให้ในตัว ได้มุม Euler (heading/pitch/roll) และ Quaternion สำหรับขับโมเดล 3D โดยตรง ตั้งแต่เวอร์ชัน 0.1.6 แยกอัตราได้ 2 ชั้น: อัตราป้อน Fusion กับอัตราสุ่มตัวอย่าง ปรับอิสระจากกัน

Field	ความหมาย	หน่วย	ช่วงค่า	ตัวหารค่า wire
acceLX / acceLY / acceLZ	ความเร่งแกน X, Y, Z	m/s²	-20 ... 20	÷100
gyroX / gyroY / gyroZ	ความเร็วเชิงมุมแกน X, Y, Z	rad/s	-5 ... 5	÷100
temperatureC	อุณหภูมิภายในชิป	°C	-40 ... 85	÷100
headingRad	มุมทิศ (Fusion Euler)	rad	-3.15 ... 3.15	÷100
pitchRad	มุมก้ม-เงย	rad	-1.6 ... 1.6	÷100
rollRad	มุมเอียงซ้าย-ขวา	rad	-3.15 ... 3.15	÷100
quatW / quatX / quatY / quatZ	Quaternion การหมุน	-	-1 ... 1	÷10000

ช่องข้อมูล (mask bits): bit 1 · Accel bit 2 · Gyro bit 4 · Temp bit 8 · Euler bit 16 · Quaternion — ค่าเริ่มต้น mask = 31 (เปิดครบทุกช่อง)

ค่าเริ่มต้น: sampling 20 ms · publish 100 ms · โหมด periodic · ถือว่าข้อมูลขาด (stale) เมื่อเขียนเกิน 500 ms · ย่านใช้งานทั่วไป: ความเร่ง ±2 m/s², การหมุน ±1 rad/s

5.2 BMM350 — สนามแม่เหล็ก (เข็มทิศ)

Field	ความหมาย	หน่วย	ช่วงค่า	ตัวหารค่า wire
magX / magY / magZ	ความเข้มสนามแม่เหล็กแกน X, Y, Z	µT	-1000 ... 1000	÷100
temperatureC	อุณหภูมิภายในชิป	°C	-40 ... 85	÷100

ช่องข้อมูล: bit 1 · Magnetometer bit 2 · Temp — ค่าเริ่มต้น mask = 3 (เปิดทั้งคู่)

ค่าเริ่มต้น: sampling 1000 ms · โหมด periodic · delta 0.10 · ช่วงขั้วต่ำ 50 ms · stale 3000 ms · สนามแม่เหล็กโลกปกติอยู่ราว ±100 µT

5.3 SHT40 — อุณหภูมิและความชื้น

Field	ความหมาย	หน่วย	ช่วงค่า	ตัวหารค่า wire
temperatureC	อุณหภูมิอากาศ	°C	-40 ... 125	÷100
humidityPct	ความชื้นสัมพัทธ์	%RH	0 ... 100	÷100

ช่องข้อมูล: bit 1 · Temp bit 2 · Humidity — ค่าเริ่มต้น mask = 3

ค่าเริ่มต้น: sampling 1000 ms · โหมด periodic · delta 0.10 · หน่วงขั้นต่ำ 50 ms · stale 3000 ms

5.4 DPS368 — ความดันบรรยากาศ

Field	ความหมาย	หน่วย	ช่วงค่า	ตัวหารค่า wire
pressureHpa	ความดันบรรยากาศ	hPa	300 ... 1200	÷10
temperatureC	อุณหภูมิภายในชิป	°C	-40 ... 85	÷100

ช่องข้อมูล: bit 1 · Pressure bit 2 · Temp — ค่าเริ่มต้น mask = 3

ค่าเริ่มต้น: sampling 1000 ms · โหมด periodic · delta 0.10 · หน่วงขั้นต่ำ 50 ms · stale 3000 ms ·

ความดันระดับน้ำทะเลปกติอยู่ราว 900–1100 hPa

ค่า wire คืออะไร? บนสาย UART ค่าจะถูกส่งเป็นจำนวนเต็ม (เช่น อุณหภูมิ ×100) เพื่อประหยัดแบนด์วิดท์ — ทุกช่องทางอ่านค่าปกติ (UI, SDK) จะหารกลับให้เป็นค่าจริงพร้อมหน่วยแล้วโดยอัตโนมัติ คอลัมน์ "ตัวหารค่า wire" มีไว้สำหรับผู้ที่ถอดเฟรมดิบเองเท่านั้น

6 กล้องและงาน Vision

คำตอบสั้น: บนตัวบอร์ดไม่มีกล้อง — เซนเซอร์บนบอร์ดมีเพียง 4 ตัวตามหัวข้อ 5

อย่างไรก็ตาม ตัวโปรแกรมมีระบบกล้องในฝั่งซอฟต์แวร์ (ใช้เว็บแคมของคอมพิวเตอร์) ซึ่งเปิดใช้ในรุ่น

PRO+ เท่านั้น — รุ่น minimal-sensor ที่ติดตั้งอยู่นี้ยังปิดความสามารถนี้ไว้:

- **Camera Input** — โหลดจับภาพจากเว็บแคม (ต้องกดอนุญาตสิทธิ์กล้องก่อน) ส่งออกเป็นสัญญาณ Video bus + ค่าสุภาพสตรีม
- **Video Texture / Material Video / CSS3D Camera Feed** — นำภาพวิดีโอไปแปะบนวัตถุ 3D หรือแสดงเป็นแผงลอยในฉาก
- **MediaPipe Vision** — วิเคราะห์ภาพในเครื่อง (ไม่ส่งขึ้นคลาวด์):

Vision Pose ตรวจท่าทางร่างกาย

Vision Hands ตรวจมือ

Vision Face ตรวจใบหน้า

Vision Object ตรวจจับวัตถุ (EfficientDet Lite0)

Landmarks Debug
- รุ่น PRO ขึ้นไปยังมีโหมดเสียง เช่น **Microphone** (ไมค์คอมพิวเตอร์ ต้องอนุญาตสิทธิ์เช่นกัน)

สรุป: ต้องการภาพจากกล้อง → ใช้เว็บแคมผ่านโหนด Camera Input (รุ่น PRO+) · ต้องการตรวจจับการเคลื่อนไหว/ทิศทางของบอร์ด → ใช้ BMI270 + BMM350 ที่มีในทุกรุ่น

7 การเชื่อมต่ออุปกรณ์

เปิดเมนู ≡ ในแอปเพื่อจัดการวิธีที่อุปกรณ์คุยกับโปรแกรม หรือใช้ **Open Connection Panel** ซึ่งพาตั้งค่าที่ละขั้น USB → Wi-Fi → MQTT → Live

ช่องทาง	ใช้เมื่อ	รายละเอียดทางเทคนิค
Serial (USB)	ต่อสาย USB กับบอร์ดโดยตรง	UART ความเร็ว 921600 bps (รองรับ 9600–921600) ผ่าน Serial Bridge ที่ถอดรหัสเฟรม BS2
Wi-Fi	บอร์ดอยู่บนเครือข่ายไร้สายเดียวกัน	ตั้งค่าผ่าน Connection Panel (เฟิร์มแวร์รองรับ capability WIFI/BLE)
MQTT	ส่งข้อมูลผ่าน broker บน LAN หรือคลาวด์	โปรแกรมมี broker ในตัว (สั่ง Start MQTT Broker) — TCP :1883, WebSocket :8883; หัวข้อ (topic) แยกตาม MAC ของบอร์ดได้
Simulator	ไม่มีบอร์ด	ดูหัวข้อ 14

สถานะสะพานเชื่อมต่อได้ที่แถบสถานะล่างของ VS Code หรือสั่ง **Bitstream Studio: Bridge Status**

8 วิธีเรียกอ่านค่าเซนเซอร์

อ่านได้ 4 ทาง เรียงจากง่ายไปขั้นสูง — ทุกทางใช้ข้อมูลชุดเดียวกันจาก Serial Bridge

8.1 อ่านจากหน้าจอ (ไม่ต้องเขียนโค้ด)

1. เปิด **Sensor Telemetry** → ค่าทุกเซนเซอร์ขึ้นที่ telemetry deck แบบเรียลไทม์ พร้อมเกจ กราฟ และประวัติการหมุน 3D
2. หรือใน **Sensor Studio** ลากโหนดเซนเซอร์ (เช่น *SHT40* → *Temperature*) ต่อเข้ากับโหนดแสดงผล (*Radial Gauge, Plotter, Numeric Display*)

8.2 อ่านด้วยโค้ดผ่าน TelemetryClient (แนะนำสำหรับงานเซนเซอร์)

Telemetry Provider คือ API เซนเซอร์อย่างเป็นทางการที่ `ws://127.0.0.1:9997` มี client แบบ typed ให้ใน SDK ค่าที่ได้ถูกแปลงเป็นหน่วยจริงแล้ว (`fields` + `units`)

ขั้นแรก ส่งออก SDK ด้วยคำสั่ง **Bitstream Studio: Export Live-Data SDK (npm package)** แล้วติดตั้ง:

```
npm install ./bitstream-live-data-0.1.0.tgz
```

```
import { TelemetryClient, SENSOR_CATALOG } from '@bitstream/live-data';

const t = new TelemetryClient();           // ค่าเริ่มต้น ws://127.0.0.1:9997
await t.connect();

// แ่็ตดลือกเซนเซอร์ทั้ง 4 ตัว ถูกส่งมาให้ทันทีที่เชื่อมต่อ
t.on('catalog', (c) => console.log(c.sensors.map(s => s.label)));
t.on('connection', (c) => console.log(c.state, c.route)); // uart | simulator

// อ่านค่ารายเซนเซอร์ (typed)
t.onSensor('sht40', (s) => {
  console.log(s.fields.temperatureC, '°C /', s.fields.humidityPct, '%RH');
});
t.onSensor('bmi270', (s) => {
  console.log('accel', s.fields.accelX, s.fields.accelY, s.fields.accelZ);
});
t.onSensor('dps368', (s) => console.log('pressure', s.fields.pressureHpa, 'hPa'));
t.onSensor('bmm350', (s) => console.log('mag', s.fields.magX, s.fields.magY, s.fields.magZ));

// ตั้งค่าเซนเซอร์จากโค้ด
await t.command('sensor.cfg.set', { sensor: 'sht40', publishIntervalMs: 500 });
```

เหตุการณ์ (event) ที่รับฟังได้: `catalog` `config` `sample` `connection` `hello` `stale` `response` — และเรียก `t.request('catalog' | 'config' | 'connection')` เพื่อขอข้อมูลซ้ำได้ทุกเมื่อ ใช้ได้ทั้งในเบราว์เซอร์และ Node ≥ 21

หมายเหตุ: field ทุกตัวเป็น optional — จะมีเฉพาะช่องที่เปิดไว้ใน publish mask เท่านั้น (ดู mask bits ในหัวข้อ 5) และ `SENSOR_CATALOG` ในแพ็คเกจมีป้ายชื่อ/หน่วย/ช่วงค่าครบทุก field สำหรับสร้างแดชบอร์ด

8.3 อ่านผ่าน LiveDataClient (WebSocket T3D หรือ MQTT)

สำหรับงานที่กว้างกว่าเซนเซอร์ (topic อื่นๆ, สั่งงาน actuator, เชื่อมหลายแอป) ใช้ `LiveDataClient`

— API เดียวสลับ transport ได้:

```
import { LiveDataClient, DEFAULT_T3D_WS_URL } from '@bitstream/live-data';

const client = new LiveDataClient({
  transport: 'websocket',          // หรือ 'mqtt'
  url: DEFAULT_T3D_WS_URL,        // ws://127.0.0.1:9998
  identity: { role: 'webapp', name: 'my-dashboard' },
});

// อ่านอีเวนต์เซนเซอร์ดิบจากบอร์ด
await client.subscribe('bitstream2/evt/sensor', (msg) => {
  console.log(msg.topic, msg.payload); // payload ถูก parse เป็น JSON ให้แล้ว
});

// ส่งคำสั่งของแอปเราเอง
await client.publish('app/commands/led', { on: true });
```

สลับเป็น MQTT เพียงเปลี่ยนสองบรรทัด:

```
const client = new LiveDataClient({
  transport: 'mqtt',
  url: 'ws://127.0.0.1:8883',      // หรือ mqtt://127.0.0.1:1883 (เฉพาะ Node)
});
```

รองรับ wildcard แบบ MQTT (+, #) ทั้งสอง transport, QoS 0–2, ช่องข้อมูล binary และ retain (เฉพาะ MQTT)

8.4 อ่านผ่าน MQTT client ภายนอก (เครื่องมืออะไรก็ได้)

เมื่อสั่ง **Start MQTT Broker** แล้ว ใช้เครื่องมือ MQTT มาตรฐานเชื่อมต่อได้เลย เช่น mosquitto:

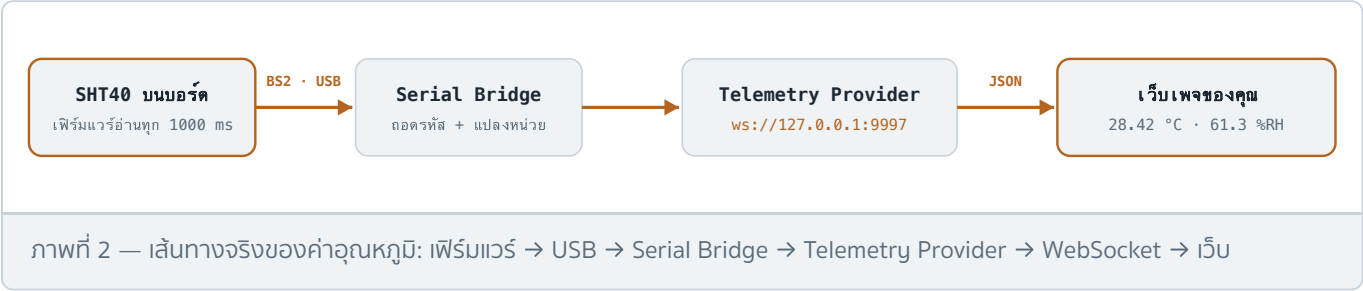
```
mosquitto_sub -h 127.0.0.1 -p 1883 -t 'bitstream2/evt/sensor' -v
```

9 Implement จริง: ส่งข้อมูลจากบอร์ดขึ้นเว็บ

ทำเว็บเพจของตัวเองให้แสดงค่าเซนเซอร์สดจากบอร์ด — ยกตัวอย่างด้วยเซนเซอร์อุณหภูมิ SHT40 ตั้งแต่ต้นจนจบ

9.1 ข้อมูลวิ่งผ่านอะไรบ้าง

บอร์ดไม่ได้คุยกับเว็บเพจโดยตรง — เฟิร์มแวร์อ่านค่า SHT40 ตามรอบ sampling แล้วส่งขึ้นสาย USB เป็นเฟรม BS2 (UART 921600 bps) ฟังก์คอมพิวเตอรื **Serial Bridge** ใน Bitstream Studio จะถอดรหัสเฟรม แปลงเป็นค่าจริงพร้อมหน่วย แล้วกระจายให้บริการบนเครื่อง เว็บเพจของคุณแค่เปิด WebSocket ไปที่บริการเหล่านั้น:



เลือกช่องทางเชื่อมตามสถานการณ์:

สถานการณ์	ช่องทางที่แนะนำ
เว็บรันบนเครื่องเดียวกับ VS Code (เดโม, แดชบอร์ดส่วนตัว)	Telemetry Provider :9997 — ง่ายสุด ได้ค่าแปลงหน่วยแล้ว
หลายแอปบนเครื่องเดียวกัน / อยาก pub-sub topic อีสาะ	T3D broker :9998 หรือ MQTT :8883
เว็บอยู่คนละเครื่อง / บนอินเทอร์เน็ต	MQTT ผ่าน broker กลาง — ตั้งค่า MQTT ของบอร์ด/โปรแกรมให้ publish ขึ้น broker ที่ทุกฝั่งเข้าถึงได้ แล้วเว็บ subscribe จาก broker นั้น (บริการ 9997/9998 ผูกกับ 127.0.0.1 ใช้ข้ามเครื่องไม่ได้)

9.2 เตรียมระบบ (ทำครั้งเดียวต่อเซสชัน)

- เสียบบอร์ดผ่าน USB
- Command Palette → **Bitstream Studio: Start All Backend Services** (หรืออย่างน้อย **Start Serial Bridge**)
- toolbar → **Bitstream** → **Link** — เห็นค่าขึ้นใน Telemetry deck ก่อน แปลว่าท่อบข้อมูลพร้อมแล้ว

9.3 ตัวอย่างที่ 1 — เว็บเพจเดียวจบ ไม่ต้องติดตั้งอะไรเลย

Telemetry Provider พุด JSON ตรง ๆ บน WebSocket ธรรมดา กันที่ที่เชื่อมต่อกันจะส่ง catalog, config, connection แล้วตามด้วย `bitstream:sample` ทุกครั้งที่บอร์ด publish — ตัวอย่าง sample ของ SHT40 ที่วิ่งมาจริง:

```
{
  "type": "bitstream:sample",
  "v": 1,
  "payload": {
    "sensor": "sht40", "sensorId": 2, "counter": 1234,
    "deviceMs": 56789, "hostMs": 1783123456789,
    "origin": "uart", "mask": 3,
    "fields": { "temperatureC": 28.42, "humidityPct": 61.3 },
    "units": { "temperatureC": "°C", "humidityPct": "%RH" }
  }
}
```

บันทึกไฟล์นี้เป็น [index.html](#) แล้วเปิดในเบราว์เซอร์ได้เลย (ดับเบิลคลิก หรือเสิร์ฟผ่านหัวข้อ 9.5):

```

<!doctype html>
<html lang="th">
<meta charset="utf-8">
<title>อุณหภูมิห้อง - SHT40</title>
<body style="font-family:sans-serif; text-align:center; padding-top:10vh">
  <h1>อุณหภูมิห้อง</h1>
  <div id="temp" style="font-size:64px; font-weight:bold"></div>
  <div id="hum" style="font-size:24px; color:#666"></div>
  <div id="status" style="margin-top:24px; font-size:14px; color:#999">กำลังเชื่อมต่อ...</div>

<script>
const $ = (id) => document.getElementById(id);
const ws = new WebSocket('ws://127.0.0.1:9997'); // Telemetry Provider

ws.onopen = () => { $('status').textContent = 'เชื่อมต่อโปรแกรมแล้ว รอข้อมูลบอร์ต...'; };

ws.onmessage = (ev) => {
  const msg = JSON.parse(ev.data);

  // ค่าเซนเซอร์เข้ามา - สนใจเฉพาะ SHT40
  if (msg.type === 'bitstream:sample' && msg.payload.sensor === 'sht40') {
    const f = msg.payload.fields;
    if (f.temperatureC !== undefined)
      $('temp').textContent = f.temperatureC.toFixed(2) + ' °C';
    if (f.humidityPct !== undefined)
      $('hum').textContent = 'ความชื้น ' + f.humidityPct.toFixed(1) + ' %RH';
  }

  // สถานะบอร์ต (เสียบ/หลุด และมาจาก uart หรือ simulator)
  if (msg.type === 'bitstream:connection') {
    const c = msg.payload;
    $('status').textContent = c.state === 'connected'
      ? 'บอร์ตออนไลน์ (' + c.route + '): ' : 'บอร์ตหลุดการเชื่อมต่อ';
  }

  // ข้อมูลขาดหาย (บอร์ตเจียบเกิน staleAfterMs)
  if (msg.type === 'bitstream:stale' && msg.payload.sensor === 'sht40') {
    $('status').textContent = 'ไม่ได้รับค่า SHT40 มาระยะหนึ่ง...';
  }
};

ws.onclose = () => { // โปรแกรม/บริการปิด - ลองใหม่อัตโนมัติ
  $('status').textContent = 'หลุดจากโปรแกรม กำลังลองใหม่...';
  setTimeout(() => location.reload(), 2000);
};
</script>
</body>
</html>

```

9.4 ตัวอย่างที่ 2 — โปรเจกต์จริงด้วย SDK (typed, จัดการ reconnect ให้)

เหมาะสำหรับเว็บแอปที่มี bundler (Vite, React ฯลฯ): สั่ง **Export Live-Data SDK (npm package)** แล้ว

```
npm install ./bitstream-live-data-0.1.0.tgz
```

```
import { TelemetryClient } from '@bitstream/live-data';

const t = new TelemetryClient(); // ws://127.0.0.1:9997 + auto-reconnect
await t.connect();

t.onSensor('sht40', (s) => {
  const temp = s.fields.temperatureC;
  if (temp !== undefined) {
    document.querySelector('#temp').textContent =
      `${temp.toFixed(2)} ${s.units.temperatureC}`; // "28.42 °C"
  }
});

// ปรับให้บอร์ดส่งอุณหภูมิถี่ขึ้น (ทุก 500 ms) จากโค้ดเลยก็ได้
await t.command('sensor.cfg.set', { sensor: 'sht40', publishIntervalMs: 500 });
```

9.5 เสิร์ฟเว็บและทดสอบ

1. สั่ง **Bitstream Studio: Serve Web App Folder over HTTP** → เลือกไฟล์เดอร์ที่มี `index.html` → เปิด `http://127.0.0.1:8899` (พอร์ตตั้งได้ใน `ternion.webAppServer.port`)
2. จับบอร์ดอุ่นด้วยมือ — อุณหภูมิบนเว็บควรขยับขึ้นภายในราว 1 วินาที (ค่าเริ่มต้น SHT40 ส่งทุก 1000 ms)
3. ไม่มีบอร์ด? สั่ง **Start Bitstream Simulator** → toolbar เลือก **Simulator** → Link — โค้ดเดิมใช้ได้ทันที (sample จะติดป้าย `origin: "sim"`)

9.6 ทางเลือก MQTT (ข้ามเครื่อง / ต่อกับระบบอื่น)

ถ้าเว็บหรือระบบปลายทางไม่ได้อยู่เครื่องเดียวกัน ให้ใช้ MQTT: สั่ง **Start MQTT Broker** แล้ว subscribe topic `bitstream2/evt/sensor` ด้วยไลบรารี MQTT ใดก็ได้ เช่น `mqtt.js`:

```
import mqtt from 'mqtt';

const c = mqtt.connect('ws://127.0.0.1:8883'); // เนราร์เซอร์ใช้ WebSocket
// จากเครื่องอื่น: ไปที่ IP ของเครื่องที่รัน VS Code หรือใช้ broker กลางที่ทุกฝั่งเข้าถึงได้

c.on('connect', () => c.subscribe('bitstream2/evt/sensor'));
c.on('message', (topic, buf) => {
  const evt = JSON.parse(buf.toString()); // อีเวนต์เซนเซอร์ติบจากบอร์ด
  console.log(topic, evt); // ดูโครงสร้างจริงก่อน แล้วค่อยกรอง sht40
});
```

เฟิร์มแวร์เองก็มีความสามารถ MQTT ในตัว (capability `MQTT` + client id อัตโนมัติจาก MAC, topic แบบ `bitstream2/<MAC>/sensors`) — ตั้งค่าผ่าน **Open Connection Panel** ขั้นตอน MQTT เพื่อให้ข้อมูลขึ้น broker กลางโดยตรงสำหรับงาน IoT จริงหลายอุปกรณ์

เช็กลิสต์เมื่อเว็บไม่ขึ้นค่า: ① Telemetry deck ในโปรแกรมเห็นค่าไหม (ถ้าไม่ → ตรวจสอบ Bridge/Link ตามหัวข้อ 17) ② คอนโซลเนราร์เซอร์ต่อ `ws://127.0.0.1:9997` ได้ไหม ③ เซนเซอร์ถูก disable หรือ mask ปิดช่องอยู่หรือเปล่า (หัวข้อ 10)

10 การตั้งค่าเซนเซอร์และคำสั่งควบคุม

10.1 พารามิเตอร์ตั้งค่า (ใช้ได้กับเซนเซอร์ทุกตัว)

พารามิเตอร์	ชนิด	ช่วงค่า	ความหมาย
<code>enabled</code>	boolean	–	ปิดแล้วเฟิร์มแวร์จะไม่สตรีมเซนเซอร์ตัวนั้นเลย
<code>publishMode</code>	enum	–	<code>periodic</code> ส่งตามรอบเวลา · <code>on_change</code> ส่งเมื่อค่าเปลี่ยนเกินเกณฑ์ · <code>hybrid</code> ใช้ทั้งสองเงื่อนไข
<code>mask</code>	u8	0 ... 255	bitmask เลือกช่องข้อมูลที่จะส่ง (ดู mask bits ของแต่ละเซนเซอร์ในหัวข้อ 5)
<code>samplingIntervalMs</code>	u16	10 ... 60000	คาบการสุ่มตัวอย่างของเฟิร์มแวร์ (ms); 0 = ไม่มี tick
<code>publishIntervalMs</code>	u16	0 ... 60000	คาบการส่งขึ้น UART (ms); 0 = ส่งทุกครั้งที่สุ่ม
<code>deltaX100</code>	u16	0 ... 10000	เกณฑ์การเปลี่ยนแปลง $\times 0.01$ สำหรับโหมด on_change / hybrid (เช่น 10 = เปลี่ยน 0.10 จึงส่ง)
<code>minPublishIntervalMs</code>	u16	0 ... 60000	เวลาขั้นต่ำระหว่างการส่งสองครั้ง (กันส่งถี่เกิน)

10.2 ตั้งจากหน้าจอ

เวิร์กสเปซ **Sensor Telemetry** → แพลงตั้งค่าเซนเซอร์ → ปรับค่า → กด **Apply** การกด Apply จะอัปเดตร่างการตั้งค่า (configuration draft) ในเครื่อง และส่งลงเฟิร์มแวร์เมื่อลิงก์กับบอร์ดอยู่ (การโปรแกรมอุปกรณ์ขึ้นสูงอาจใช้เครื่องมือเฟิร์มแวร์ควบคู่)

10.3 ตั้งจากโค้ด — คำสั่งของ Telemetry Provider

คำสั่ง	หน้าที่	อาร์กิวเมนต์
<code>sensor.cf</code> <code>g.get</code>	อ่านการตั้งค่าปัจจุบัน	—
<code>sensor.cf</code> <code>g.set</code>	เขียนการตั้งค่า	<code>{ sensor, enabled?, publishMode?, mask?, samplingIntervalMs?, publishIntervalMs?, deltaX100?, minPublishIntervalMs? }</code>
<code>bmi270.mod</code> <code>e.get</code>	อ่านโหมด BMI270 (fusion feed / sample rate)	—
<code>bmi270.mod</code> <code>e.set</code>	ตั้งโหมด BMI270	payload โหมดสองอัตรา

```
// ตัวอย่าง: ให้ SHT40 ส่งเฉพาะเมื่ออุณหภูมิ/ความชื้นเปลี่ยนแปลง 0.25 แต่ไม่ถี่กว่า 200 ms
await t.command('sensor.cfg.set', {
  sensor: 'sht40',
  publishMode: 'on_change',
  deltaX100: 25,
  minPublishIntervalMs: 200,
});
```

11 MQTT / WebSocket Topics

หัวข้อหลักในเนมสเปซ `bitstream2/*` — ใช้ subscribe ผ่าน LiveDataClient หรือ MQTT client ใดก็ได้

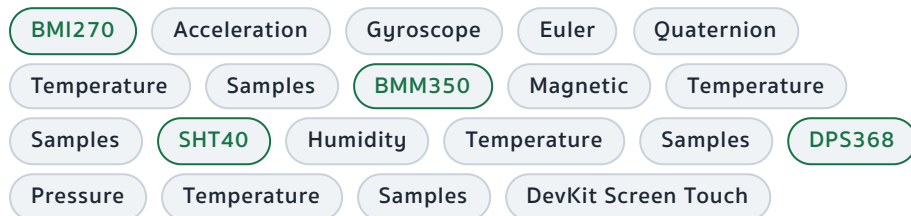
Topic	ข้อมูลที่ไหลผ่าน
<code>bitstream2/hello</code>	การจับมือ / ระบุตัวตนของเซสชัน
<code>bitstream2/evt/sensor</code>	อีเวนต์ค่าเซนเซอร์ (topic หลักสำหรับอ่านค่า)
<code>bitstream2/evt/actuator</code>	อีเวนต์ actuator เช่น LED
<code>bitstream2/evt/status</code>	สถานะอุปกรณ์
<code>bitstream2/evt/diag</code>	ข้อมูลวินิจฉัย
<code>bitstream2/evt/ui</code>	อีเวนต์ UI ของ HMI (เช่นการคลิก)
<code>bitstream2/req</code> · <code>bitstream2/res</code>	คำขอ / คำตอบ
<code>bitstream2/metrics</code>	เมตริกระบบ
<code>bitstream2/dev/status</code>	สถานะฟังก์ชันอุปกรณ์
<code>bitstream2/dev/telemetry-provider/status</code>	สถานะ Telemetry Provider
<code>bitstream2/telemetry/route</code>	เส้นทางข้อมูลที่ใช้อยู่ (uart / simulator)
<code>bitstream2/sensor/stream-policy-updated</code>	แจ้งเมื่อนโยบายสตรีมเปลี่ยน
<code>bitstream2/dev/sim/control</code> · <code>.../sim/state</code> · <code>.../sim/wave/set</code>	ควบคุม / สถานะ / ตั้งรูปคลื่นของ Simulator
<code>bitstream2/sim/host-tx</code> · <code>bitstream2/sim/status</code>	ฝั่งส่งของโฮสต์จำลอง / สถานะจำลอง
<code>bitstream2/dev/inject-rx</code>	ฉีดเฟรม RX เพื่อทดสอบ
<code>bitstream2/dev/virt/config</code> · <code>.../config/ack</code>	ตั้งค่าอุปกรณ์เสมือน + ตอบรับ
<code>bitstream2/hmi-ui/scene</code>	ข้อมูลจาก HMI
<code>t3d/broker/monitor</code>	มอนิเตอร์ข้อความของ T3D broker

เนมสเปซรุ่นเก่า (legacy) ที่ยังพบได้: `bitstream/dev/telemetry`, `bitstream/probe/telemetry` และ `bitstream/<MAC>/sensors` (แยก topic ตาม MAC address ของบอร์ด)

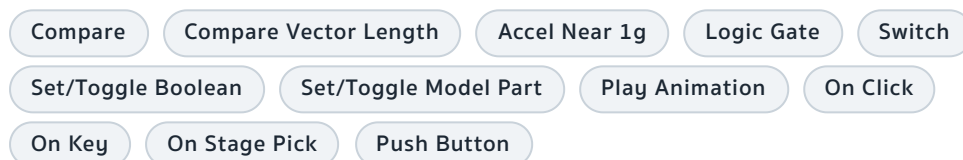
12 คลังโหนดใน Sensor Studio

โหนดทั้งหมดที่มีในตัวโปรแกรม จัดกลุ่มตามครอบครัว (family) — บางครอบครัวเปิดใช้เฉพาะรุ่น PRO / PRO+ ดูหัวข้อ 16

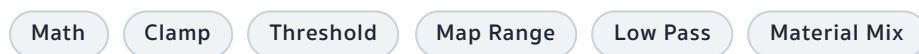
เซนเซอร์ (sensor)



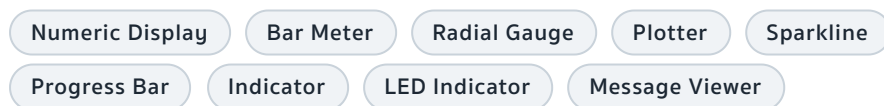
ตรรกะและเหตุการณ์ (logic)



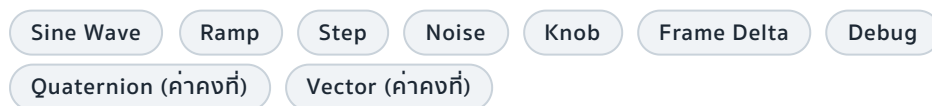
คณิตศาสตร์และการแปลงสัญญาณ (math / transform)



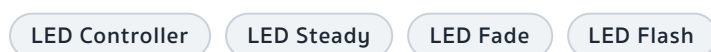
ตัวแสดงผล (output / indicator)



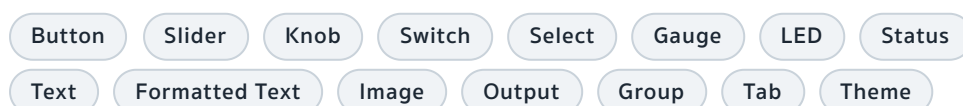
ตัวกำเนิดสัญญาณ (generator)



Actuator (สั่งงานฮาร์ดแวร์)



แดชบอร์ด (dashboard)



การเชื่อมต่อ (connectivity)

MQTT Publisher

MQTT Subscriber

WebSocket Publisher

WebSocket Subscriber

เสียง (audio — รุ่น PRO ขึ้นไป)

Microphone

Oscillator

Audio SFX

Audio Machine

Audio File Player

Audio Scope

Audio Output

วิดีโอและ Vision (รุ่น PRO+)

Camera Input

Video Texture

Material Video

CSS3D Camera Feed

Vision Pose

Vision Hands

Vision Face

Vision Object

Vision Landmarks Debug

ฉาก 3D (scene)

Camera / View

Camera Switch

Environment

Scene Output

Scene Settings

Scene Time

Light

Model Source

Model Viewer

Box · Sphere · Plane · Cylinder · Cone · Torus · Capsule

Material: Basic · Standard · Physical · Toon · Normal

GLB Material Color/Property/Texture

Part Transform

Animation Clip · Blend · Mix · Merge · Clips

Part Spin

3D Rotation (Euler / Quaternion)

ยูทิลิตี้ (utility)

ค่าคงที่ Number / Float / Integer / Boolean

Math

Average

Normalize

Scalar/Vector Lerp

Vector: Add · Scale · Cross · Dot · Distance · Length · Length² · Angle · Project · Reject · Normalize · Split · Combine

Quaternion: Multiply · Conjugate · Inverse · Normalize · Slerp · Split · Combine · $\rightleftharpoons$ Euler · Axis-Angle · Rotate Vector

Degrees $\rightleftharpoons$ Radians

Euler Heading

Tilt from Accel

Transform from Euler/Quaternion

Position/Rotation/Scale

Object Transform / Spawner

Timer

Multiplexer

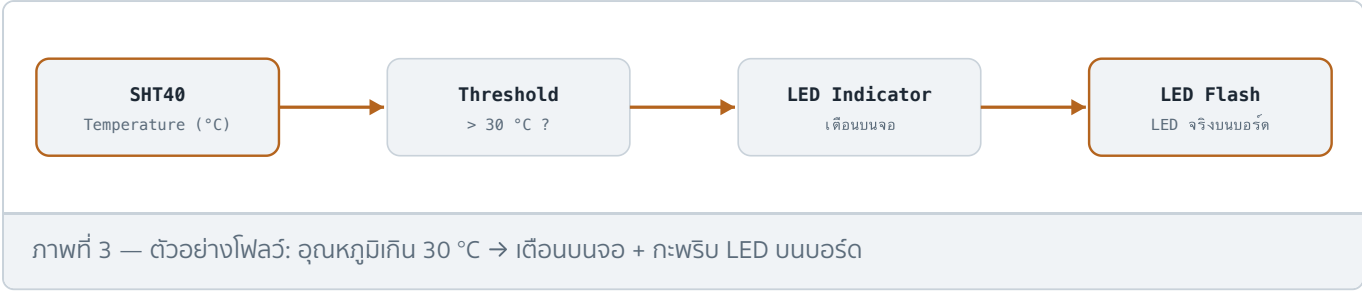
JSON Creator

Sensor Snapshot

ฟิสิกส์: Physics World · Rigid Body · Box/Sphere Collider · Fixed/Hinge Joint · IK Chain

เอฟเฟกต์: Particle Emitter · Contact Shadows · Fog · Post-Processing · UV Transform · Morph Target · Material Variant

ตัวอย่างโฟลว์พื้นฐาน



13 Digital Twin — โมเดล 3D

โมเดลไม่ได้ฟังมากับตัวติดตั้ง แต่ดาวน์โหลดอัตโนมัติครั้งแรกจากคลัง `ternion-3d-assets-free` บน GitHub

โมเดล	คำอธิบาย
TESAIoT DevKit	บอร์ดหลัก (แอสเซตบังคับสำหรับเริ่มระบบ) — มี Twin พร้อมป้าย CSS3D บอกค่าเซนเซอร์
PSoC E84 AI	บอร์ด PSoC Edge (แอสเซตบังคับ)
ABB Robot Arm	แขนกลอุตสาหกรรม
TESA Drone	โดรน
PLC Training Kit	ชุดฝึก PLC
Two-Wheels Bot	หุ่นยนต์ทรงตัวสองล้อ
Car (physics)	รถยนต์พร้อมกล่องติดตามและระบบฟิสิกส์ชน (colliders)
Healthcare Gateway	เกตเวย์อุปกรณ์การแพทย์
LiteJam RBB Guitar	กีตาร์สาธิต

จากแวลด้อม: Cubemap 5 ชุด (bridge, park, snow, storforsen, yokohama) และ HDRI 10 ชุด (apartment, city, dawn, forest, lobby, night, park, studio, sunset, warehouse)

คำสั่งจัดการแอสเซต: **Check Free Assets on Disk** (ตรวจ), **Download Free Assets from GitHub** (ดาวน์โหลด), **Reveal Assets Folder in Explorer** (เปิดโฟลเดอร์), **Set Asset Source Strategy** (เลือกนโยบาย local-only / local-first / online-only) — ถ้าแอสเซตบังคับหาย จะมีหน้าต่าง "Required assets" เด้งชวนกด *Sync assets*

14 โหมด Simulator

จำลองข้อมูลเซนเซอร์ครบทั้ง 4 ตัวโดยไม่ต้องมีฮาร์ดแวร์ เหมาะกับการเรียนการสอนและทดสอบโฟลว์:

- **อัตราอนจिन:** เลือกได้ 20 / 30 / 60 Hz
- **รูปคลื่น:** sine, square, triangle, sawtooth, ramp, pulse (คลื่นฐาน sine 0.2 Hz แต่ละเซนเซอร์เหลื่อมเฟสกันเล็กน้อยเพื่อให้ดูสมจริง)

- ควบคุมผ่าน topic: `bitstream2/dev/sim/control` (สั่ง), `.../sim/state` (สถานะ), `.../sim/wave/set` (ตั้งรูปคลื่น)
- ทุก sample จากตัวจำลองติดป้าย `origin: "sim"` แยกจากข้อมูลบอร์ดิ้งจริงชัดเจน

เริ่ม/หยุดด้วยคำสั่ง **Start / Stop Bitstream Simulator** แล้วเลือก **Simulator** ที่ toolbar → **Link**

15 คำสั่ง การตั้งค่า คีย์ลัด และพอร์ต

15.1 คำสั่งทั้งหมดใน Command Palette (หมวด "Bitstream Studio")

กลุ่ม	คำสั่ง	หน้าที่
เปิดใช้งาน	Open Bitstream Studio	เปิดหน้าต่างหลักของแอป
	Choose Application	เลือกเวิร์กสเปซ (Telemetry / Studio)
	Open ... (Sensor Telemetry tab)	เปิดตรงไปที่เวิร์กสเปซ Telemetry
	Open ... (Sensor Studio tab)	เปิดตรงไปที่เวิร์กสเปซ Studio
บริการเบื้องหลัง	Start / Stop Serial Bridge	เปิด/ปิดสะพานเชื่อม Serial ↔ WebSocket
	Start / Stop MQTT Broker	เปิด/ปิด MQTT broker ในเครื่อง
	Start / Stop Bitstream Simulator	เปิด/ปิดตัวจำลองข้อมูล
	Start All Backend Services	เปิดทุกบริการพร้อมกัน
	Stop All Backend Services / Shutdown Backend	หยุดทุกบริการ
	Bridge Status	ดูสถานะสะพานเชื่อมที่แถบสถานะ
การเชื่อมต่อ	Open Connection Panel	ตัวช่วยตั้งค่า USB → Wi-Fi → MQTT → Live
	Open Setup Checklist	รายการตรวจสอบการติดตั้งครั้งแรก
เว็บแอป / SDK	Export Live-Data SDK (npm package)	ส่งออกแพ็คเกจ SDK ไว้ใช้ในโปรเจกต์ของคุณ
	Serve Web App Folder over HTTP / Stop Web App Server	เสิร์ฟไฟล์เดอร์เว็บแอปที่พอร์ต 8899
	Open in Browser (Dev Server)	เปิดแอปในเบราว์เซอร์ภายนอก
	Set Local Web App Port (Browser)	กำหนดพอร์ตของเซิร์ฟเวอร์ภายใน
แอสเซต 3D	Check Free Assets on Disk	ตรวจสอบว่าโมเดลครบหรือไม่
	Download Free Assets from GitHub	ดาวน์โหลด/ซิงก์โมเดล 3D
	Run Download Free Assets in Terminal (CLI)	ดาวน์โหลดผ่านเทอร์มินัล
	Open Free Assets Loader / Open Model Loader	หน้าต่างการโมเดล
	Reveal Assets Folder in Explorer	เปิดไฟล์เดอร์แอสเซตบนดีสก์
	Set Asset Source Strategy	นโยบายแหล่งโมเดล (local-first ฯลฯ)
อื่น ๆ	Refresh / Reload Bitstream Studio Panel	รีเฟรชหน้าเว็บวิว
	Reload VSCode Window	รีโหลดทั้งหน้าต่าง
	Toggle Developer Tools	เครื่องมือดีบักรีวิว
	Open Tools Panel	แผงเครื่องมือเสริม

15.2 คีย์ลัด

ปุ่ม	ทำงาน	เงื่อนไข
<code>Cmd</code> + <code>/</code> (macOS) · <code>Ctrl</code> + <code>/</code> (Windows)	เปิด/ปิด Quick Actions	โฟกัสอยู่ในหน้าแอป
<code>Q</code>	เปิด/ปิด Q-menu (เมนูตามตำแหน่งเมาส์ในฉาก 3D)	โฟกัสอยู่ในหน้าแอป

15.3 การตั้งค่า (Settings → ค้นหา "ternion")

คีย์	ค่าเริ่มต้น	ความหมาย
<code>ternion.ws.brokerPort</code>	9998	พอร์ต WebSocket broker ฟังก์ชัน Serial (T3D)
<code>ternion.ws.modelBrokerPort</code>	9999	พอร์ต broker ของ Model Loader / แคร็ดดาไลก์โมเดล
<code>ternion.ws.aiBrokerPort</code>	9987	พอร์ต AI bridge (ยังปิดในรุ่นนี้)
<code>ternion.backend.autoStart</code>	true	เริ่มบริการเบื้องหลังอัตโนมัติเมื่อเปิด VS Code (ถ้าพอร์ตว่าง)
<code>ternion.webAppServer.port</code>	8899	พอร์ตเซิร์ฟเวอร์ HTTP สำหรับเว็บแอปผู้ใช้ (0 = สุ่มพอร์ต)
<code>ternion.browserApp.port</code>	0	พอร์ตเซิร์ฟเวอร์ภายในสำหรับ Open in Browser (0 = สุ่มพอร์ต)
<code>ternion.mqtt.configuration</code>	null	โปรไฟล์การเชื่อมต่อ MQTT + ประวัติการ publish
<code>ternion.githubToken</code>	""	โทเคน GitHub (สิทธิ์อ่านอย่างเดียวพอ) เพิ่มลิมิตดาวน์โหลดแอสเซต
<code>ternion.assets.sourceStrategy</code>	local-first	นโยบายแหล่งโมเดล: local-only / local-first / online-only

15.4 สรุปพอร์ตทั้งหมดบนเครื่อง

พอร์ต	บริการ	โปรโตคอล	ใช้ทำอะไร
9997	Telemetry Provider	WebSocket + JSON	API เซนเซอร์หลัก — แคร็ดดาไลก์ ค่าเซนเซอร์ การตั้งค่า
9998	T3D Broker	WebSocket (T3D)	pub/sub ทั่วไป, topic ดิบ, QoS 0–2, binary
9999	Model Broker	WebSocket	โหลดโมเดล 3D / แคร็ดดาไลก์แอสเซต
8883	MQTT Broker	MQTT over WebSocket	สำหรับเบราว์เซอร์
1883	MQTT Broker	MQTT TCP	สำหรับ Node / เครื่องมือ MQTT ทั่วไป
9987	AI Bridge	WebSocket	LLM + MCP tools (ปิดอยู่ในรุ่นนี้)
8899	Web App Server	HTTP	เสิร์ฟเว็บแอป/เดโมของผู้ใช้

16 รุ่นของโปรแกรม (Tier)

รุ่น	สิ่งที่ได้
BASIC	Sensor Telemetry + Sensor Studio (ไฟพลั๊กคัส) — ไม่มีเสียง, ไม่มี Vision, ไม่มี Stage 3D เต็มรูป/Dashboard พิเศษ
PRO	BASIC + คอนทริบิวต์เสียง (Microphone, Oscillator, Audio bus ฯลฯ)
PRO+	PRO + กล้อง/MediaPipe Vision (Pose, Hands, Face, Object) + Screen Composer

แพ็คเกจที่ติดตั้งอยู่นี้ใช้โปรไฟล์ **minimal-sensor**: เปิดเฉพาะ Sensor Telemetry + Sensor Studio ส่วนโมดูล Course Studio, HMI Studio, Simulation Data, Screen Composer ถูกปิดไว้ (มีเกมเพลตติดมากับแพ็คเกจแล้ว รอเปิดใช้ในรุ่นที่สูงขึ้น เช่น HMI ขนาดจอ 4.3"/7": sensors-monitor, drone-controller, robot-controller, industrial-plc, ev-charge, smart-farm, smart-home-climate, healthcare-gateway)

17 การแก้ปัญหา

อาการ	วิธีแก้
ไม่มีข้อมูลสดเข้ามา	สั่ง Start Serial Bridge · ตรวจสอบสาย/พอร์ต USB · Reload Window
WebSocket / bridge error	สั่ง Start Serial Bridge แล้วกด Link ใหม่
ภาพ 3D ว้างเปล่า	รอดาว์โหลดครั้งแรกให้จบ หรือสั่ง Sync Free Pack to Disk / Download Free Assets from GitHub
ค่าเซนเซอร์ค้าง มีป้าย stale	ข้อมูลเขียนเกินกำหนด (BMI270: 500 ms, ตัวอื่น: 3000 ms) — ตรวจสอบการเชื่อมต่อ หรือดูว่าเซนเซอร์ถูก disable ไว้หรือไม่
คอนโซลเตือน <code>removeChild</code>	มักไม่มีผลหลังเปิดครั้งแรก — Reload Window
ส่วนขยาย activate ไม่ขึ้น	ติดตั้งใหม่จาก Marketplace
ดาว์โหลดแอสเซตติดลิมิต GitHub	ใส่โทเคนใน <code>ternion.githubToken</code> (สักริอ่านอย่างเดียว)
พอร์ตชนกับโปรแกรมอื่น	แก้พอร์ตใน Settings (หัวข้อ 15.3) แล้ว Reload Window

18 โปรโตคอล BS2 (สำหรับผู้ใช้งานขั้นสูง)

ข้อมูลระหว่างบอร์ดกับโปรแกรมวิ่งบนโปรโตคอลเฟรมชื่อ **BS2 (bitstream2)** ผ่าน UART 921600 bps

- ชนิดอีเวนต์:** `EVT_SENSOR` (ค่าเซนเซอร์) · `EVT_ACTUATOR` · `EVT_STATUS` · `EVT_DIAG` · `EVT_UI` / `EVT_CLICK` · `EVT_STATE` · `EVT_DONE`
- กลุ่มคำสั่ง:** คำสั่งระบบ (เช่น `SYSTEM_REBOOT` = cmdId 0x03, `SENSOR_CFG`, `FUSION_FEED_SET`), คำสั่ง LED (`BS2_LED_CMD` — ความสว่าง ๐-16 ขั้นตอน 0.01%, 10000 = 100%), คำสั่ง MQTT (`BS2_MQTT_CMD` — ส่งรับ client id อัตโนมัติจาก MAC), คำสั่ง HMI และโปรไฟล์โมดูล
- Capability bits ของเฟิร์มแวร์:** `BLE_PERIPH` · `BMI` · `HMI_UI` · `LED` · `MQTT` · `SCENE_YXZ` · `SENSOR_STREAM_POLICY` · `TIME` · `WIFI` — โปรแกรมอ่านคำนี้ตอน HELLO เพื่อรู้ว่าจะทำอะไรได้

- คำสั่งที่เฟิร์มแวร์ไม่รองรับจะตอบกลับ `BS2_COMMAND_UNSUPPORTED`
- ทดสอบโดยไม่มีบอร์ด: จัดพรอมผ่าน topic `bitstream2/dev/inject-rx` ได้

19 แหล่งอ้างอิง

- Marketplace: marketplace.visualstudio.com/items?itemName=TERNIONDEV.bitstream-studio
- ซอร์สโค้ด: github.com/drsanti/Bitstream-Studio
- คลังโมเดล 3D ฟรี: github.com/drsanti/ternion-3d-assets-free
- เอกสาร SDK ในเครื่อง: สั่ง **Export Live-Data SDK** แล้วอ่าน `README.md` ในแพ็คเกจ
- สัญญาอนุญาต: MIT

คู่มือนี้เรียบเรียงจากการวิเคราะห์แพ็คเกจ bitstream-studio-0.1.7.vsix โดยตรง (README, changelog, แคริตาล็อกเชนเซอร์ใน Live-Data SDK, manifest และโค้ดภายใน) — ข้อมูล ณ วันที่ 11 กรกฎาคม 2026